

verify-ping

User manual

Version 0.16.5 · 2026-10-05

<https://github.com/atlanteg/verify-ping>

verify-ping — user manual

`verify-ping` measures what a link really does to your packets: **loss, reordering and latency, attributed to each direction separately**. Every probe carries a unique, SHA-256-verified payload, so a counted reply is always the reply to *that* probe, intact. It also finds which source ports land on the fastest ECMP paths, and can pick and hold the fastest live flow.

One Python file, no dependencies, Linux or macOS, Python 3.9+.

Contents

1. [Install](#)
2. [Quick start](#)
3. [How it works](#)
4. [Loss test \(client mode\)](#)
5. [Server](#)
6. [Reverse mode `-R`](#)
7. [Path hunt `--hunt`](#)
8. [Roulette `--roulette`](#)
9. [Pinning a flow and watching it over time](#)
10. [One-way delay and clocks](#)
11. [Load estimate and confirmation](#)
12. [Troubleshooting](#)
13. [Option reference](#)
14. [Limits and wire format](#)

1. Install

On **every** host involved (clients and servers), the same release:

```
curl -fsSL https://raw.githubusercontent.com/atlanteg/verify-ping/v0.16.1/verify_ping.py -o
verify_ping.py.tmp \
  && mv verify_ping.py.tmp verify_ping.py && python3 verify_ping.py --version
```

Requirements: Python 3.9+; root (`sudo`) only for raw TCP and ICMP. Both sides print their version on start-up and the client warns when the server's differs — the wire and control formats only work between equal releases.

`python3 verify_ping.py --help` is a complete, grouped reference with examples.

2. Quick start

Remote host (server) — one process serves every protocol on the same ports; pick a port range *below* the ephemeral range (see §12):

```
sudo python3 verify_ping.py --check-ports --port 20100 -P 8          # are they free?
sudo python3 verify_ping.py --server --protocol all --port 20100 -P 8
```

Local host (client) — 3000 probes of 500 bytes at 10 pkt/s over UDP:

```
python3 verify_ping.py 10.0.0.1 --protocol udp --port 20100 -c 3000 -i 0.1 -s 500
```

The client first prints the load it will generate and waits for Enter (§11), then runs, then prints loss and reordering **per direction** (§4).

Open the firewall / security group for the port range in **both UDP and TCP** toward the server.

3. How it works

Streams and ports. A client run has one or more *streams* (`-P`), each on its own server port (`--port`, `--port+1`, ...) and its own source port. Routers hash the 5-tuple onto parallel paths, so different streams may take different paths — this is what `--hunt` exploits.

Verified payload. Every probe is `header + SHA-256 expansion of the header`: unique per packet, verifiable on return. A corrupted or mixed-up reply counts as `bad_payload`, never as success.

Echo with a stamp. The server echoes each probe after writing a small *stamp* into it: its receive counter (`rseq`, the order probes arrived) and its receive time (monotonic and wall-clock). The stamp region is zeroed again before verification, so the SHA-256 check still covers the whole packet.

Direction attribution. The server keeps, per stream, the *arrival log* (which sequence numbers arrived, in what order). After the run the client pulls it **in-band over the same socket** — no second port, works through NAT — and reconciles:

Figure	Meaning
forward loss	sent – reached the server (dropped on the way <i>to</i> the server)
reverse loss	reached the server – verified (echo dropped on the way <i>back</i>)
forward reorder	sequence inversions in the server's arrival order
reverse reorder	inversions of <code>rseq</code> in the client's reply order (against what the server actually sent)

One-way delay. From the stamps: `forward = server_rcv - client_send`, `reverse = client_rcv - server_rcv`. With monotonic clocks these carry an unknown but *constant* offset, so flows can be compared exactly (`--hunt`); with wall clocks that both sides report as synchronised they are absolute (§10).

Protocols.

-- protocol	What it is	Loss/reorder per direction	One-way delay	Needs
udp	UDP datagrams	yes	yes	nothing
tcp	raw TCP segments (no handshake)	yes	yes	root both sides; fails through stateful NAT
tcp- stream	real TCP connections, framed	no (TCP hides them)	yes	nothing; works through NAT
icmp	echo request/reply	no (kernel answers)	round trip only	root on the client
all	client: icmp,udp,tcp- stream,tcp; server: everything			

4. Loss test (client mode)

```
python3 verify_ping.py 10.0.0.1 --protocol udp --port 20100 -P 4 -c 3000 -i 0.1 -s 500
```

Output, section by section:

```
--- 10.0.0.1:20100..20103 verified udp statistics ---
streams=4 count_per_stream=3000 sent=12000 verified=11968 lost=32 bad_payload=0 loss=0.267%
duplicates=0 unexpected=0
checked_payload=5.7MB elapsed=304.1s
server version 0.16.1 (same as client)
stream=1 sent=3000 verified=2990 lost=10 loss=0.333% bad_payload=0
...
missing request indexes: 1:44, 1:94, 2:109, ...
```

- `verified` — replies that matched their probe byte for byte; `lost` — no reply within `-W`; `bad_payload` — a reply that did not match; `duplicates` / `unexpected` — extra or foreign replies.
- Exit status is 0 only when everything was verified.

```
--- directional statistics (server arrival log fetched in-band after the run) ---
all streams: sent=12000 reached_server=12000 verified=11968
all streams: loss: forward=0 (0.000%) reverse=32 (0.267%) total=32 (0.267%)
all streams: reorder: forward=0 reverse=0 end_to_end=0 forward_dup=0
stream=1 reverse-lost seqs (echo never returned): 44, 94, 109, ...
```

- `reached_server` — probes the server logged. Here every probe arrived and 32 echoes were dropped on the way back: the loss is on the **reverse** path.
- A stream with `reached_server=0` gets an explicit nothing reached the server on this port: `forward path blocked (firewall / security group?)`. Its log is fetched through the other streams' sockets, so a blocked port still gets a verdict.
- Lost sequence numbers tell burst from random loss: long runs of consecutive numbers mean an outage; scattered numbers mean a policer or queue.

```
--- one-way delay ---
clocks: client synced, est. error ±0.083 ms (chrony); server synced, est. error ±0.008 ms
(kernel)
forward (client -> server): min 10.787 ms p50 10.9 ms | reverse (server -> client): min 12.
457 ms p50 12.6 ms (valid within ±0.091 ms)
asymmetry (forward - reverse, by minima): -1.670 ms
```

Printed when both clocks are synchronised with a usable error estimate; see §10.

Progress: a line every 100 verified replies (`--progress N`, 0 silences; `-v` prints every reply). `--series SEC` adds a per-window line (§9).

5. Server

```
sudo python3 verify_ping.py --server --protocol all --port 20100 -P 8
```

- Serves `udp`, `tcp` (raw), `tcp-stream` and the `icmp` observer at once, one thread each, on the same port numbers (UDP and TCP are separate port spaces). `--protocol udp,tcp-stream` picks a subset. Root is needed for raw TCP and the ICMP observer; without it the ICMP observer says so and the ICMP test still works (the kernel answers echo).
- If one server cannot start (port busy, no root for raw TCP) the process reports it and exits with status 1.
- The log names the protocol on every line and announces each run:

```
verify_ping v0.16.1 server: udp, tcp, tcp-stream, icmp on port(s) 20100..20107
[14:02:10] udp: test traffic from 10.0.0.7 started
[14:02:11] udp: rx=50 pkt/s streams=4 clients=1 total=50
[14:02:13] udp: finished, 100 packets over 4 stream(s) from 10.0.0.7
[14:02:31] tcp-stream: connection from 10.0.0.7:41822 on port 20100
[14:02:33] tcp-stream: 10.0.0.7:41822 closed, 58.6KB echoed
[14:02:40] icmp (observed; kernel answers): test traffic from 10.0.0.7 started
```

`--progress 0` silences it. The server keeps `-R` sessions for an hour so a slow client can still fetch its results.

Ports. Check before starting, and stay below the ephemeral range

(`/proc/sys/net/ipv4/ip_local_port_range`, usually 32768–60999): an outgoing connection may grab a port in that range at any moment and the bind fails with "Address already in use" although nothing listens there. The server says so when it happens.

```
python3 verify_ping.py --check-ports --port 20100 -P 8
```

6. Reverse mode `-R`

Like `iperf3 -R`: the client still **initiates** (so it works from behind NAT; only the server needs an open port), then the roles swap — the server sends the probes and the client echoes them. The report is printed on the client with the directions relabelled: `forward = server → client`.

```
python3 verify_ping.py 10.0.0.1 --protocol udp --port 20100 -c 3000 -i 0.1 -s 500 -R
```

`-R` does **not** change who connects. If host A can reach B but not the other way round, the client always runs on A: without `-R` A probes B, with `-R` B probes A through the path A opened. UDP only. A stream whose start the server never acknowledges (port blocked toward the server, or an old server) is reported and skipped.

7. Path hunt `--hunt`

Find which flows (source/destination port combinations) land on the fastest paths — in each direction separately.

```
sudo python3 verify_ping.py 10.0.0.1 --protocol all --port 20100 -P 8 --hunt 64 -c 50 -i 0.2 -s 500
```

- `--hunt N` runs N flows at once, each with its own source port and a destination port from the server's `-P` range (ICMP varies the identifier). Flows are phase-shifted across one interval so a round never leaves as one burst.
- With several protocols they run one after another and a cross-protocol summary follows.

Per protocol:

```
--- path hunt: udp, 64 flows x 50 probes ---
flow path  src->dst      rtt_min  rtt_p50  fwd_rel  rev_rel  ok/sent  loss  lost
f/r
 34 A      50618->20101    26.976  27.053   +3.725   +0.000   50/50    0.00%
0/0
 37 B      59677->20104    27.743  27.843   +0.000   +4.493   50/50    0.00%
0/0
...
paths by rtt_min (tolerance 0.15 ms):
 A: 26.976 ms  flows [34]
...
best round trip: flow 34 (50618->20101) 26.976 ms, path A
forward (client -> server): 3 level(s) [+0.000 ms x2, +3.027 ms x18, +6.0 ms x10] +2 outlier(s), spread 6.97 ms; fastest: flows [37]
reverse (server -> client): 2 level(s) [+0.000 ms x4, +4.233 ms x58] +2 outlier(s), spread 5.56 ms; fastest: flows [34]
no flow is fastest both ways (asymmetric ECMP)
clocks: ... -> absolute one-way (valid within ±0.09 ms): best forward 10.787 ms (flow 37), best reverse 12.457 ms (flow 34)
```

How to read it:

- `rtt_min` is the propagation floor of a flow's path pair; `rtt_p50` the typical value.
- `fwd_rel` / `rev_rel` — one-way delay of this flow **relative to the best flow in that direction** (exact, no clock sync needed).
- **Levels:** flows are grouped by *gaps* — a new level starts where the sorted minima leave a gap wider than `--hunt-tolerance` (0.15 ms). Each level is shown as its offset from the best with the number of flows on it: `+3.027 ms x18` means 18 flows were 3 ms slower in that direction. A level needs at

least two flows; single flows are outliers. A level whose values spread continuously is flagged as queueing rather than distinct paths — more probes per flow (`-c`) sharpen the minima.

- `fastest: flows [...]` — the flows on the fastest level in that direction; the next line says whether any flow is fastest both ways. `within 0.5 ms of the best: forward N, reverse M, both K` answers the practical question when a continuous spread has been split into several levels.
- `lost f/r` — the per-flow loss split (UDP, raw TCP).

The summary names the best round trip per protocol and overall, and the level structure per direction. ICMP is ranked by round trip only.

8. Roulette `--roulette`

For when the far side assigns the **return** path per flow with a timeout rather than by a stable hash: a fast reverse path that only a few percent of new flows get, and that an idle flow loses after some minutes. No source port can be pinned in advance; what works is what a latency-sensitive application should do anyway — connect many, measure, keep the best, never let it idle.

```
sudo python3 verify_ping.py 10.0.0.1 --roulette 100 --port 20100 -P 8 -c 30 -i 0.2 -s 500 -  
-keep 1 --hold 1800 --series 30
```

- Runs for **UDP and tcp-stream side by side** by default, concurrently, each with its own N flows; lines are prefixed `[udp] / [tcp]`. `--protocol` picks one. A UDP "flow" is the connected socket, kept alive by the probes.
- Opens N flows over the `-P` ports, probes each `-c` times, ranks them by **forward + reverse minima** (the forward path of an open flow cannot be changed, so the kept flow must be the best round trip), prints the top ten (absolute one-way columns when clocks are synced), the level structure per direction and how many flows are fast both ways.
- Keeps the best `--keep` flows, **closes the rest**, and holds the kept ones — one probe per `-i` each — with a `--series` line per window (default 30 s) until `--hold` seconds or Ctrl+C, then prints min / p50 / max per direction over the hold. A path change on a live flow shows as a step in `rev`.

Automatic re-roll. The return path can change on a *live* flow (seen in practice after minutes of continuous traffic), so keeping a flow busy does not guarantee it stays fast. With `--reroll` the hold phase watches each window's `rtt_min` against the held flow's baseline; after `--reroll-windows` (2) consecutive windows more than `--reroll-threshold` (2 ms) above it, a fresh batch of N flows is opened and ranked, the held flow is swapped for the best candidate if that is at least 0.5 ms better (otherwise the candidates are closed and the new level becomes the baseline), and the event is logged. The summary then lists every held segment, each re-roll with before/after, the share of time spent within the threshold of the best `rtt_min` seen, and the mean/p50 round trip over the whole hold — i.e. what a "connect many, keep the best, re-roll on degradation" strategy actually achieves on this path.

```
sudo python3 verify_ping.py 10.0.0.1 --roulette 100 --port 20100 -P 8 -c 30 -i 0.2 -s 500 -  
-hold 7200 --series 30 --reroll
```

```
[tcp] --- roulette tcp: 100 flows ranked by forward + reverse minima ---
[tcp] rank flow src->dst      rtt_min  rev_rel  fwd_rel  rev_abs  fwd_abs
[tcp]   1   91 37152->20102    23.527   +0.000   +0.189   12.073   11.431
...
[tcp] forward levels: 3 level(s) [+0.000 ms x53, +3.694 ms x25, +6.278 ms x20]; 53 of 100 f
lows (53.0%) on the fastest
[tcp] reverse levels: 3 level(s) [+0.000 ms x36, +1.282 ms x26, +4.674 ms x36]; 36 of 100 f
lows (36.0%) on the fastest
[tcp] fast in both directions: 21 of 100 flows (flows [91, 26, ...])
[tcp] keeping flow 91 (37152->20102); closed 99 others
[tcp] holding for 1800s, one probe every 0.2s per flow, window 30s
[tcp] [14:38:23] rtt min 23.527 p50 23.787 ms | fwd min 11.431 | rev min 12.073 ms (180/18
0 replies)
```

9. Pinning a flow and watching it over time

`--src-port` binds stream 1 to a fixed source port (stream N gets `PORT+N-1`) for udp, tcp and tcp-stream, so an exact 5-tuple the hunt found can be measured again later:

```
python3 verify_ping.py 10.0.0.1 --protocol tcp-stream --port 20103 -P 1 --src-port 43438 -c
200 -i 0.05 -s 500
```

The header confirms `pinned source port(s) 43438`; the one-way block says whether both directions are still on the fast levels.

`--series SEC` prints, every SEC seconds, the window's `rtt min/p50`, the one-way minima and the replies/sent count — in any client mode. Combine with a long run to see *when* a path changes:

```
python3 verify_ping.py 10.0.0.1 --protocol tcp-stream --port 20103 -P 1 --src-port 43438 -c
36000 -i 0.05 --series 30
```

```
[11:42:03] rtt min 28.407 p50 28.512 ms | fwd min 11.523 | rev min 16.822 ms (600/600 repl
ies)
```

Re-running the same TCP 4-tuple within a minute fails with "Cannot assign requested address": the previous connection is in `TIME_WAIT`. Wait, or allow reuse on the client with `sysctl -w net.ipv4.tcp_tw_reuse=1` (safe for outgoing connections). The tool names this when it happens.

10. One-way delay and clocks

Absolute one-way delays need both clocks disciplined, and the tool checks that itself rather than trusting you:

- Each side reads the kernel's view of its clock (`adjtimex`: the `STA_UNSYNC` flag and the error estimate `ntpd / ptp4l` maintain), else `chronyc tracking` (`loffsetl` + root dispersion + root delay / 2), else the kernel's max error while it is under 10 ms; `timedatectl` as a last resort. The server reports its status in-band; the client prints both.

- Absolute figures are printed only when both sides are synced **with** an estimate and the combined bound is at most half the round trip: (valid within ± 0.091 ms). Otherwise the reason is named — server clock NOT synced, no error estimate from the time daemon (chrony and ntpd provide one, systemd-timesyncd does not), clock error bound ± 966 ms is larger than the delays being measured — and `--wallclock` prints the figures anyway, marked UNVERIFIED.
- A one-way delay that comes out negative, or a direction below 20% of the round trip (0.6 ms back on a 32 ms path), is not believable and proves the clocks disagree more than they admit. The tool then withdraws the absolute figures everywhere (one-way block, hunt, roulette columns) and names the minimum disagreement — a daemon can report ± 0.1 ms while being 16 ms off; check `chronyc tracking` and `chronyc sources -v` on that host. `--series` lines mark such values as (clock offset, not absolute).
- On a virtual machine the reported error is relative to the hypervisor's clock (PHC0 / `/dev/ptp_hyperv`, Amazon Time Sync), not to UTC: a VM following a host that is 16 ms off reports ± 5 μ s. To check a host against the outside world without changing its sync, add measurement-only sources and read them after a few minutes:

```
sudo chronyc add server time.google.com iburst nselect sudo chronyc add server
time.cloudflare.com iburst nselect sleep 600; chronyc sources -v; chronyc sourcestats -v sudo
chronyc delete time.google.com; sudo chronyc delete time.cloudflare.com
```

Milliseconds in their `Last sample / Offset` columns while the reference clock claims microseconds means the host clock is wrong (redploy the VM to another host, or stop trusting the reference clock alone).

What is "synced enough": NTP over the internet gives 1–10 ms (too coarse for sub-millisecond asymmetry); chrony against a nearby stratum-1, or a cloud time service, 0.01–1 ms; PTP/GPS microseconds. On Ubuntu, replace `systemd-timesyncd` with `chrony` to get an estimate at all: `apt install chrony && chronyc tracking`.

Relative comparison between flows (`fwd_rel`, `rev_rel`) never needs any of this.

11. Load estimate and confirmation

Before a client sends anything it prints the load the run will generate — packets/s and bit/s **per direction** (the echo mirrors it the other way), including Ethernet/IP/transport overhead, per protocol and phase, with the peak for sequential phases or the sum for concurrent roulettes — and waits:

```
verify_ping v0.16.1 load estimate (per direction; the echo adds the same the other way):
udp roulette, probing 100 flows          500.0 pkt/s      2.17 Mbit/s
udp roulette, holding 1 flow(s)          5.0 pkt/s       21.68 kbit/s
tcp roulette, probing 100 flows          500.0 pkt/s      2.23 Mbit/s
tcp roulette, holding 1 flow(s)          5.0 pkt/s       22.32 kbit/s
all roulettes probing at once            1000.0 pkt/s     4.40 Mbit/s
all roulettes holding at once            10.0 pkt/s      44.00 kbit/s
probing takes ~9s, then holding 1800s
Proceed? [Enter = yes, Ctrl+C = abort]
```

Ctrl+C there aborts with nothing sent. `-y` skips the question (scripts); so does a non-terminal stdin, with a note.

Rough sizing: $\text{pkt/s} = \text{flows} / \text{interval}$; $\text{bit/s} \approx \text{pkt/s} \times (\text{size} + 42) \times 8$.

Loss tests need many probes — resolution is $1/\text{probes}$ — so lower their load with a larger `-i` or a smaller `-s`, not with fewer probes.

Latency work (hunt, roulette, pinned flows) does not: it needs a solid *minimum* per flow, which 10–15 probes give, and packet size and rate are irrelevant to it. Light settings — 100-byte payload, one probe per second per flow — cut the load by 20x with no loss of information:

```
# roulette with re-roll: ~100 pkt/s (115 kbit/s) per protocol for 15 s per round, 1 pkt/s while holding
sudo python3 verify_ping.py 10.0.0.1 --roulette 100 --port 20100 -P 8 -c 15 -i 1 -s 100 --hold 7200 --series 60 --reroll

# path hunt, all protocols: ~64 pkt/s (73 kbit/s) per protocol for 18 s each
sudo python3 verify_ping.py 10.0.0.1 --protocol all --port 20100 -P 8 --hunt 64 -c 15 -i 1 -s 100

# watch one pinned flow for an hour at 1 pkt/s
python3 verify_ping.py 10.0.0.1 --protocol tcp-stream --port 20103 -P 1 --src-port 43438 -c 3600 -i 1 -s 100 --series 60
```

With 15 probes a flow's minimum may sit 0.1–0.3 ms above its true floor (queueing); the `within 0.5 ms` of the `best` line absorbs that, and `-c 30` sharpens it if needed.

12. Troubleshooting

Symptom	Meaning	Do
nothing reached the server on this port: forward path blocked	packets to that port never arrive	open the port (UDP and TCP) in the firewall / security group; <code>--check-ports</code> on the server
<code>tcp connect ... timed out</code>	nothing came back at all	port filtered, or no host; ICMP in <code>--protocol all</code> runs first and says whether the host is reachable
<code>tcp connect ... Connection refused</code>	host answered, nothing listens	start the server / check its port range
raw <code>tcp:0</code> replies while <code>tcp-stream</code> works	stateful NAT drops segments without a SYN	use <code>tcp-stream</code> from behind NAT
server never acknowledged the <code>-R start</code>	this host cannot reach the server, or old server	<code>-R</code> does not change who connects: run the client on the side that can reach the other
Address already in use on a port nothing listens on	port is in the ephemeral range, taken by an outgoing connection	pick <code>--port</code> below the ephemeral range (e.g. 20100)
Cannot assign requested address with <code>--src-port</code>	4-tuple still in TIME_WAIT	wait 60 s or <code>sysctl -w net.ipv4.tcp_tw_reuse=1</code>
arrival log fetch failed / incomplete	old server, or path too lossy	same version both sides; raise <code>-W</code>
server version X differs from client Y	releases differ	reinstall the same release on both sides
absolute one-way delay withheld: ...	a clock is not synced or has no estimate	§10
no output at all	an empty file: <code>curl -f</code> left 0 bytes on a bad URL	re-download via a temp file as in §1
client prints nothing between runs	<code>--progress 0</code> , or <code>--hunt / --roulette</code> (progress off by default)	<code>--progress N</code> or <code>--series SEC</code>

13. Option reference

Option	Default	Meaning
host		destination (client modes)
--protocol P[,P..]	icmp	icmp, udp, tcp, tcp-stream, all; lists with --hunt, --roulette, --server
--server		run the echo side
--hunt N	0	path hunt with N flows
--roulette N	0	open N udp + N tcp-stream flows, keep the best, hold
-R		reverse roles (udp)
--check-ports		test the port range for udp and tcp, exit
-p, --port	random (server)	first port; -P consecutive follow
-P, --parallel	1	streams (client) / ports (server); under hunt/roulette: ports to spread over
-c, --count	3000	probes per stream (≤ 65535)
-i, --interval	0.08	seconds between probes per stream
-s, --size	1200	payload bytes (≥ 70)
-W, --timeout	3	seconds to wait for late replies
--src-port PORT		pin the source port of stream 1
--bind ADDR	0.0.0.0	server bind address
--hunt-tolerance MS	0.15	gap that separates path levels
--keep K	1	flows to keep per protocol in roulette
--hold SEC	until Ctrl+C	roulette hold duration
--reroll		roulette: replace a degraded held flow from a fresh batch
--reroll-threshold MS	2.0	window <code>rtt_min</code> above baseline by more than this = degraded
--reroll-windows N	2	degraded windows in a row that trigger a re-roll
--series SEC	0	per-window latency line
--progress N	100 / 0	client progress every N replies; server rx line (0 off)
-v		print every verified reply
--no-directional		skip the in-band fetch
--wallclock		print absolute one-way even when unverified
-y, --yes		skip the load confirmation
--version		print the version

14. Limits and wire format

- Sequence numbers are 16-bit: at most 65535 probes per stream per run.
- Payload at least 70 bytes (client header + server stamp).
- `--hunt` / `--roulette` need the server's `-P` to cover the destination ports they spread over; flows wrap around the range.
- In-band control (arrival logs, clock and version exchange, `-R` start) travels as chunks of about 1 KB on the test sockets themselves and is re-requested until complete; nothing else needs to be opened.
- Wire format: magic `vpng4` (client header: ident, stream, seq, index, monotonic and wall-clock send time, run nonce; server stamp: `rseq`, monotonic and wall-clock receive time), control magic `vpnc4`. Run the same release on both sides.
- Test helpers for development live in `tests/` (a lossy relay with known drops and swaps per direction, and a blocked-port relay).

License: MIT.